ShadowXLab — Kubernetes Hands-On Labs Workbook

12 enterprise labs · beginner to enterprise · for ShadowXLab Kubernetes program (Module 3).

Table of Contents

Lab 01 — Install Kubernetes on Ubuntu with kubeadm (Beginner, 90m)

Lab 02 — Troubleshooting Kubernetes Like an SRE (Engineer, 75m)

Lab 03 — Kubernetes Internal Workflow — From kubectl to runc (Engineer, 60m)

Lab 04 — Pods, Deployments & Self-Healing Rollouts (Beginner, 60m)

Lab 05 — Services, Selectors, Endpoints & DNS (Engineer, 60m)

Lab 06 — Flannel, CNI & Pod Networking (Engineer, 60m)

Lab 07 — Resources, Requests, Limits & OOMKilled (Engineer, 45m)

Lab 08 — Persistent Storage — PV, PVC & StorageClass (Engineer, 45m)

Lab 09 — RBAC, Secrets, ServiceAccounts & NetworkPolicies (Enterprise, 60m)

Lab 10 — Monitoring & Observability — Prometheus, Grafana, Loki (Enterprise, 75m)

Lab 11 — Hybrid Cluster — Joining a Windows Worker Node (Enterprise, 90m)

Lab 12 — Enterprise Incident Investigation Capstone (Enterprise, 120m)

Lab 01 — Install Kubernetes on Ubuntu with kubeadm

ID: K8S-LAB-01 · **Category:** Beginner · **Level:** Beginner · **Duration:** 90 min

Focus: kubeadm, kubelet, kubectl, containerd, Flannel, control plane, worker node

Objective

Stand up a single-node Kubernetes cluster on Ubuntu using kubeadm, containerd and Flannel, then deploy a Pod that proves the cluster works end-to-end.

Concepts Covered

• Why swap must be disabled for the kubelet

• Kernel modules and sysctl required by CNI

• Container runtime (containerd) vs Docker Engine

• What kubeadm, kubelet and kubectl each actually do

• PodCIDR and overlay networking (Flannel)

Prerequisites

• Ubuntu 22.04 LTS with sudo

• 2 vCPU, 2 GB RAM, 20 GB disk minimum

• Outbound internet to registry.k8s.io and ghcr.io

Architecture

kubeadm bootstraps the static pods of the control plane on the host. The kubelet on every node speaks to containerd (the CRI), which uses runc to start Linux containers. Flannel runs as a DaemonSet and programs an overlay so every Pod gets a routable IP from 10.244.0.0/16.

Lab Steps

01. Update Ubuntu and disable swap

Kubelet refuses to start when swap is on because the scheduler's memory accounting becomes incorrect once pages can be paged out.

```
sudo apt-get update && sudo apt-get -y upgrade
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab
```

Expected: swap entries commented in /etc/fstab; `free -h` shows Swap: 0B

Internals: Cgroups v1/v2 memory limits assume RSS == working set. With swap enabled, OOMKill decisions become non-deterministic.

02. Load kernel modules and sysctl for CNI

br_netfilter lets iptables see bridged traffic so Service/Pod NAT rules apply. ip_forward enables Linux to route between Pod networks.

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay && sudo modprobe br_netfilter

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sudo sysctl --system
```

Expected: `lsmod | grep br_netfilter` shows the module loaded.

Internals: kube-proxy programs iptables/IPVS rules on the host. Without bridge-nf-call-iptables those rules are skipped for bridged pod traffic and Services break silently.

03. Install containerd and configure systemd cgroup driver

Kubelet and containerd MUST agree on the cgroup driver. Kubernetes ≥1.22 defaults to systemd; mismatches cause subtle pod restarts.

```
sudo apt-get -y install containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd && sudo systemctl enable containerd
```

Expected: `sudo systemctl status containerd` → active (running).

Internals: containerd exposes the CRI socket at /run/containerd/containerd.sock. kubelet calls CRI RPCs (RunPodSandbox, CreateContainer) which containerd implements via runc.

04. Install kubeadm, kubelet and kubectl

kubeadm bootstraps the cluster, kubelet is the node agent, kubectl is the CLI. Pinning prevents an `apt upgrade` from breaking the control plane.

```
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.30/de
  sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

Expected: `kubeadm version` and `kubectl version --client` print v1.30.x.

Internals: kubelet runs as a systemd unit (/etc/systemd/system/kubelet.service). It will crash-loop until kubeadm writes /var/lib/kubelet/config.yaml during init.

05. Initialize the control plane

kubeadm creates the CA, writes static pod manifests for api-server/etcd/controller-mgr/scheduler, and uploads cluster config.

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16 --cri-socket=unix:///run/containerd/containerd.sock
```

Expected: Ends with `Your Kubernetes control-plane has initialized successfully!` and a `kubeadm join` command. SAVE IT.

Internals: Static pods come from /etc/kubernetes/manifests/. Kubelet watches that directory and starts the pods directly — they're not scheduled.

06. Configure kubectl access

admin.conf holds the client cert for the cluster-admin user. Copy it to ~/.kube/config so kubectl can authenticate.

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl get nodes
```

Expected: Node shows `NotReady` — expected until a CNI is installed.

Internals: kubectl reads context, cluster and user blocks from ~/.kube/config and signs each request with the client cert listed in `users[].user.client-certificate-data`.

07. Install Flannel CNI

Flannel programs an overlay (VXLAN) that allocates a /24 of the PodCIDR to every node and routes Pod-to-Pod traffic across nodes.

```
kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml
```

Expected: `kubectl -n kube-flannel get pods` shows kube-flannel-ds Running on the node.

Internals: Flannel writes /run/flannel/subnet.env and a CNI conflist under /etc/cni/net.d. Without that conflist, kubelet's PLEG can't set Pods to Ready.

08. Verify node and CoreDNS

Node should turn Ready within ~60s. CoreDNS pods leave ContainerCreating once the CNI is healthy.

```
kubectl get nodes
kubectl -n kube-system get pods
```

Expected: Node STATUS=Ready, coredns-* pods Running 1/1.

Internals: CoreDNS stays in ContainerCreating until kubelet can attach it to the Pod network. That requires a working CNI binary AND a populated /etc/cni/net.d/.

09. Allow workloads on the control plane (single-node lab)

By default the control plane is tainted NoSchedule. For a single-node lab we untaint it.

```
kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

Expected: `taint "node-role.kubernetes.io/control-plane" removed`

Internals: Taints repel pods unless they carry a matching toleration. Removing the taint lets the scheduler place workload pods on this node.

10. Deploy a first Pod and prove it works

Runs an nginx Pod, shows its assigned PodIP, then proves the container actually serves traffic.

```
kubectl run web --image=nginx --port=80
kubectl get pod web -o wide
kubectl exec web -- curl -s localhost | head -5
```

Expected: Pod Running 1/1, IP from 10.244.0.0/16, curl returns `<!DOCTYPE html>`.

Internals: kubelet pulled the image via containerd → CRI ImagePull, created a sandbox (pause container) for the network namespace, then the nginx container joined it.

Common Errors

Node stuck NotReady — Cause: CNI not installed or /etc/cni/net.d empty · Fix: Apply Flannel/Calico, then `journalctl -u kubelet | tail`

CoreDNS ContainerCreating — Cause: CNI not ready yet · Fix: Wait for kube-flannel-ds Running, then `kubectl -n kube-system rollout restart deploy coredns`

Flannel CrashLoopBackOff — Cause: PodCIDR mismatch with `--pod-network-cidr` · Fix: Re-init cluster with the correct CIDR, or patch flannel ConfigMap

x509 certificate signed by unknown authority — Cause: Stale ~/.kube/config from a previous cluster · Fix: Re-copy /etc/kubernetes/admin.conf and `chown` it

Pending Pods on a fresh control plane — Cause: Control-plane taint still present · Fix: Untaint or add `tolerations` to the Pod spec

Troubleshooting

• Investigate the kubelet first: `sudo journalctl -u kubelet -f`

• Inspect runtime: `sudo crictl ps`, `sudo crictl logs <id>`

• Look at CNI: `ls /etc/cni/net.d/`, `cat /run/flannel/subnet.env`

• Validate API health: `kubectl get --raw=/readyz?verbose`

Enterprise Relevance

kubeadm is still the reference bootstrap path for on-prem and air-gapped enterprise clusters. Understanding the swap/cgroup/CNI triad makes you the engineer who recovers a stuck cluster at 2am instead of the one who pages someone else.

Validation

• `kubectl get nodes` shows Ready

• `kubectl get pods -A` shows no CrashLoopBackOff

• An nginx Pod gets a 10.244.x.x IP and responds to curl

Related Incidents & Guide Sections

Incidents: K8S-INC-3002

• Control plane & worker components

• Pod lifecycle & probes

Questions

Beginner

Q. Why must swap be disabled before kubelet starts?

A. Kubelet's memory accounting assumes pages are never paged out; swap makes OOMKill decisions non-deterministic.

Q. What is the difference between kubeadm and kubelet?

A. kubeadm is a one-shot bootstrapper that creates the cluster. kubelet is the long-running node agent that talks to the API server and the runtime.

Intermediate

Q. Why does `--pod-network-cidr=10.244.0.0/16` matter for Flannel?

A. Flannel's default config expects that exact CIDR. A mismatch causes Flannel CrashLoopBackOff and Pods that never get an IP.

Advanced

Q. Where do control-plane components actually run after `kubeadm init`?

A. As static pods, manifests in /etc/kubernetes/manifests/, started directly by kubelet without going through the scheduler.

Scenario

Q. After `kubeadm init` your node is Ready but CoreDNS is stuck ContainerCreating. Where do you look?

A. /etc/cni/net.d for an empty directory, then the CNI pod's logs. CoreDNS can't get a Pod IP without a working CNI.

Viva

Q. Walk me from `kubectl run web --image=nginx` to a running container.

A. kubectl → API server → etcd → scheduler binds Pod → kubelet on node → CRI RunPodSandbox → CNI assigns IP → CRI CreateContainer → containerd → runc → Linux namespaces/cgroups.

Lab 02 — Troubleshooting Kubernetes Like an SRE

ID: K8S-LAB-02 · **Category:** Troubleshooting · **Level:** Engineer · **Duration:** 75 min

Focus: kubectl get, describe, logs, events, journalctl, investigation methodology

Objective

Master the canonical SRE investigation workflow: get → describe → logs → events → runtime — and apply it to six recurring failure modes.

Concepts Covered

• Why describe almost always beats logs as a first step

• How Events are TTL'd and where they live

• Difference between application logs (stdout) and kubelet logs (journald)

• Cluster-wide vs namespaced views

Prerequisites

• A working cluster from Lab 1

• kubectl context set to that cluster

Architecture

Each step narrows the search space. Don't skip describe — it folds events, status conditions and last-termination state into one screen.

```
kubectl get   ->  what state is the object in?
     |
     v
kubectl describe -> events + conditions + last-state
     |
     v
kubectl logs (-p) -> what did the container say (or its previous incarnation)?
     |
     v
kubectl get events -> what did the cluster say around the same time?
     |
     v
journalctl -u kubelet / crictl ps -> node-level reality check
```

Lab Steps

01. Create a deliberately broken Pod

Reproduces an ImagePullBackOff in a controlled way so you can practice the workflow.

```
kubectl run badimg --image=nginx:doesnotexist
kubectl get pod badimg
```

Expected: STATUS=ImagePullBackOff or ErrImagePull within ~30s.

Internals: kubelet asks containerd to pull the image. containerd retries with exponential backoff and reports failure back through the CRI status.

02. Step 1 — describe

The Events section at the bottom shows the registry error verbatim.

```
kubectl describe pod badimg | tail -20
```

Expected: `Failed to pull image "nginx:doesnotexist": ... manifest unknown`

Internals: Events are stored as Event objects in the same namespace with a default TTL of 1h.

03. Step 2 — fix and verify

Pods are largely immutable, but image is one of the few fields you can patch in place.

```
kubectl set image pod/badimg badimg=nginx:1.27
kubectl get pod badimg -w
```

Expected: Status goes ContainerCreating → Running.

Internals: kubelet sees the spec diff, re-asks containerd to pull, restarts the container.

04. Reproduce CrashLoopBackOff

Forces a guaranteed crash so you can inspect previous-container logs.

```
kubectl run crasher --image=busybox --restart=Never -- /bin/sh -c 'exit 1'
```

Expected: RESTARTS climbs, STATUS becomes CrashLoopBackOff.

Internals: kubelet applies an exponential backoff (10s, 20s, 40s …) up to 5 minutes between restart attempts.

05. Step 3 — logs of the previous container

Without -p you'll get the current (not-yet-restarted) container; with -p you get the one that just died.

```
kubectl logs crasher -p
```

Expected: (empty in this trivial case — but in real apps you'd see the stack trace)

Internals: Logs are pulled from /var/log/containers/* via the CRI log endpoint.

06. Step 4 — cluster events

Gives you a cross-object timeline; invaluable when something cascades.

```
kubectl get events --sort-by=.lastTimestamp | tail -15
```

Expected: BackOff events for the crasher Pod, plus any unrelated cluster events.

Internals: kubectl get events is just `kubectl get -o wide events.v1.events.k8s.io`.

07. Step 5 — node-level reality check

When the API tells you everything is fine but pods still misbehave, drop down to the kubelet and the runtime.

```
sudo journalctl -u kubelet --since '5 min ago' | tail -30
sudo crictl ps
```

Expected: kubelet log entries about backing off, crictl shows the container states.

Internals: journald is the only place you'll see why kubelet itself failed (e.g. invalid kubeconfig, cgroup driver mismatch).

Common Errors

kubectl logs returns nothing — Cause: Container restarted; you need the previous instance · Fix: Add `-p` / `--previous`

describe shows no Events — Cause: Events older than 1h; TTL'd out · Fix: Increase --event-ttl on the API server or use kube-state-metrics + persistent store

`crictl: command not found` — Cause: Not installed on the node · Fix: `sudo apt install cri-tools` or use `nerdctl` against containerd directly

Troubleshooting

• Always start with describe; logs without context is guessing

• If the API server is slow, hit `--raw=/healthz` to confirm it's the API and not your kubectl

• When something is namespace-scoped, never forget `-n`

Enterprise Relevance

Senior SREs are not the people who know the most YAML — they're the ones who follow the same investigation loop every single time under pressure.

Validation

• You can recover a broken Pod without restarting the cluster

• You can tell whether a failure is application, scheduler, runtime or network in <5 min

Related Incidents & Guide Sections

Incidents: K8S-INC-3001, K8S-INC-3003

• Troubleshooting methodology

• CrashLoopBackOff

• ImagePullBackOff

Questions

Beginner

Q. What's the first command you run when a Pod is unhealthy?

A. kubectl describe pod <name> — events + conditions in one place.

Intermediate

Q. Why does `kubectl logs` return empty for a crashlooping pod?

A. Because the current container hasn't produced output yet; use `-p` for the previous instance.

Advanced

Q. How would you correlate a Pod's failure with what the kubelet was doing?

A. Take the Pod event timestamp, then `journalctl -u kubelet --since '<ts-30s>' --until '<ts+30s>'`.

Scenario

Q. A pod shows Running 1/1 but the app is returning 503s to clients. Where do you look?

A. Service endpoints (selector mismatch), readiness probe (passing but app broken), then app logs.

Viva

Q. Walk through your five-step kubectl investigation loop.

A. get → describe → logs (-p) → get events → kubelet/crictl on the node.

Lab 03 — Kubernetes Internal Workflow — From kubectl to runc

ID: K8S-LAB-03 · **Category:** Runtime · **Level:** Engineer · **Duration:** 60 min

Focus: api-server, scheduler, kubelet, containerd, runc, reconciliation

Objective

Trace a single `kubectl apply` through every control-plane and node component until a Linux process is running.

Concepts Covered

• Desired state vs actual state (the reconciliation loop)

• How the scheduler ranks nodes

• How kubelet uses the CRI to talk to containerd

• Where namespaces and cgroups actually get created

Prerequisites

• A cluster from Lab 1

Architecture

Every component just watches the API server and reacts. There's no central coordinator — only loops converging the actual state onto the desired state stored in etcd.

Lab Steps

01. Apply a Deployment and watch events in another terminal

You'll see Scheduled → Pulling → Pulled → Created → Started in order.

```
kubectl create deployment trace --image=nginx --replicas=1
# in another terminal:
kubectl get events --watch
```

Expected: Five events within ~5s on a warm node.

Internals: Each event is emitted by a different component: scheduler, kubelet, then runtime.

02. Inspect the scheduling decision

Shows which node was picked and which scheduler made the call.

```
kubectl describe pod -l app=trace | grep -A2 Scheduled
```

Expected: `Successfully assigned default/trace-... to <node>` by default-scheduler.

Internals: Scheduler runs predicates (filter) then priorities (score). Final winner is bound via a Pod Bind subresource update.

03. See the container on the node

Proves the abstraction: there's just a Linux process under all the YAML.

```
sudo crictl ps | grep nginx
```

Expected: A container with state Running.

Internals: crictl talks the same CRI protocol kubelet uses.

04. Inspect the cgroup

The Pod gets a cgroup slice; the container is a unit inside it.

```
sudo systemd-cgls | head -30
```

Expected: A `kubepods.slice/...` hierarchy.

Internals: cgroups enforce CPU/memory limits; without them `limits:` in the Pod spec would be advisory.

05. Watch self-healing in real time

Deleting the Pod doesn't kill the workload — the ReplicaSet observes a missing Pod and asks the API for a new one.

```
kubectl delete pod -l app=trace
kubectl get pods -l app=trace -w
```

Expected: New Pod created within ~2s.

Internals: ReplicaSet controller watches Pods matching its selector and posts a new Pod when count < replicas.

Common Errors

Pod stays Pending forever — Cause: No node matches the requests/tolerations · Fix: `kubectl describe` → Events reveal which predicate failed

Pod created twice after `kubectl delete` — Cause: User deleted the Pod but a Deployment owns the ReplicaSet · Fix: Expected behavior — delete the Deployment, not the Pod

Troubleshooting

• If the scheduler is silent, check it's running: `kubectl -n kube-system get pods | grep scheduler`

• If pods never start, watch the runtime: `sudo crictl events`

Enterprise Relevance

Understanding the loop is what makes you trust Kubernetes in production: there is no magic, only convergent controllers.

Validation

• You can name every component that touches a Pod from kubectl to runc

• You can predict whether a manual `kubectl delete pod` will respawn or not

Related Incidents & Guide Sections

Incidents: K8S-INC-3004

• Control plane & worker components

• Pod lifecycle & probes

Questions

Beginner

Q. What does the scheduler actually do?

A. Picks a node for an unbound Pod and writes a Binding back to the API.

Intermediate

Q. What ties a container back to its Pod on the node?

A. The sandbox (pause) container shares its network/IPC namespaces with all app containers in the Pod.

Advanced

Q. Why is reconciliation eventually consistent?

A. Controllers act on watched events with retries; partial failures heal on the next sync rather than being transactional.

Scenario

Q. etcd disk fills up. What's the first symptom you'll see?

A. API server starts returning 500s on writes; reads may still work briefly.

Viva

Q. Where do static pods come from?

A. Files in /etc/kubernetes/manifests/, watched directly by kubelet, not scheduled.

Lab 04 — Pods, Deployments & Self-Healing Rollouts

ID: K8S-LAB-04 · **Category:** Core · **Level:** Beginner · **Duration:** 60 min

Focus: Pods, ReplicaSets, Deployments, scaling, rolling updates, rollback

Objective

Author a Deployment, scale it, roll out a new image, roll back when it breaks, and watch self-healing in action.

Concepts Covered

• Why you almost never create raw Pods in production

• Rolling update strategy parameters

• How rollback uses the ReplicaSet history

Prerequisites

• A cluster from Lab 1

Architecture

Deployments don't manage Pods directly — they manage ReplicaSets and orchestrate the gradual switch between two of them.

```
Deployment (desired: 3, image: v1)
   |
   v
 ReplicaSet  --owns-->  Pod  Pod  Pod
   |
 (on update)
   v
 New ReplicaSet (v2)  --owns-->  Pod  Pod  Pod
   |
 Old ReplicaSet scaled to 0 (kept for rollback)
```

Lab Steps

01. Create the Deployment

Three replicas of nginx 1.25.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 3
  selector: {matchLabels: {app: web}}
  template:
    metadata: {labels: {app: web}}
    spec:
      containers:
      - name: web
        image: nginx:1.25
        ports: [{containerPort: 80}]
```

```
EOF
```

Expected: deployment.apps/web created; 3 pods Running within ~10s.

Internals: Deployment controller creates a ReplicaSet, which creates 3 Pods.

02. Scale up

Edits the ReplicaSet's `replicas` field. Two new Pods get scheduled.

```
kubectl scale deploy/web --replicas=5
kubectl get pods -l app=web
```

Expected: 5/5 ready within ~10s.

Internals: ReplicaSet controller posts new Pods until count == desired.

03. Rolling update

Replaces Pods gradually using maxSurge=25%, maxUnavailable=25% defaults.

```
kubectl set image deploy/web web=nginx:1.27
kubectl rollout status deploy/web
```

Expected: `deployment "web" successfully rolled out`

Internals: A NEW ReplicaSet is created at desired=1, then scaled up while the old is scaled down — never both at full capacity unless surge allows it.

04. Break it on purpose and roll back

Bad image → rollout stalls. `rollout undo` switches traffic back to the previous ReplicaSet.

```
kubectl set image deploy/web web=nginx:doesnotexist
kubectl rollout status deploy/web --timeout=30s || true
kubectl rollout undo deploy/web
kubectl rollout status deploy/web
```

Expected: After undo: rollout succeeds.

Internals: Deployment keeps `revisionHistoryLimit` ReplicaSets at replicas=0 specifically for this.

05. Watch self-healing

Deleting a managed Pod produces an immediate replacement.

```
POD=$(kubectl get pod -l app=web -o name | head -1)
kubectl delete $POD
kubectl get pods -l app=web -w
```

Expected: A new Pod with a different suffix appears within ~2s.

Internals: ReplicaSet controller sees count<desired and POSTs a new Pod.

Common Errors

selector mismatch on a manual edit — Cause: Edited template labels but not the selector (or vice-versa) · Fix: Selectors are immutable on Deployments — delete and recreate

Rollout stuck with maxUnavailable=0 — Cause: No spare capacity to start new Pods · Fix: Raise maxSurge or add a node

Troubleshooting

• `kubectl rollout history deploy/web` shows every revision

• `kubectl rollout undo deploy/web --to-revision=2` jumps to a specific revision

Enterprise Relevance

Rolling updates + rollback are the cheapest production safety net you can buy. Every deploy should be undoable in one command.

Validation

• You can update an image without downtime

• You can roll back a bad deploy in under 30 seconds

Related Incidents & Guide Sections

Incidents: K8S-INC-3001

• Pod lifecycle & probes

Questions

Beginner

Q. Why prefer Deployments over raw Pods?

A. Self-healing, scaling, and rollouts.

Intermediate

Q. What does maxUnavailable control?

A. How many Pods can be down at once during a rollout.

Advanced

Q. Why are Deployment selectors immutable?

A. Changing them would orphan the existing ReplicaSet's Pods, risking double-running or no-running states.

Scenario

Q. A rollout is wedged. Old pods are gone, new pods Pending. What broke?

A. Likely a scheduling constraint introduced in the new spec (resources, affinity, node selector).

Viva

Q. What chain owns a Pod created by a Deployment?

A. Deployment → ReplicaSet → Pod, tracked via ownerReferences.

Lab 05 — Services, Selectors, Endpoints & DNS

ID: K8S-LAB-05 · **Category:** Networking · **Level:** Engineer · **Duration:** 60 min

Focus: Service, ClusterIP, NodePort, kube-proxy, CoreDNS, Endpoints

Objective

Expose a Deployment via ClusterIP and NodePort, prove service discovery works through CoreDNS, and debug a deliberately-broken selector.

Concepts Covered

• How Endpoints objects bind Services to Pods

• ClusterIP vs NodePort vs LoadBalancer

• DNS records created per Service

Prerequisites

• Lab 4 deployment `web` running

Architecture

A Service is a virtual IP. kube-proxy programs DNAT rules on every node so requests to ClusterIP land on a real Pod IP.

```
Client
   |
   v
Service (ClusterIP 10.96.x.x)
   |
   v   (kube-proxy iptables/IPVS rules)
   v
Endpoints  -->  Pod  Pod  Pod  (by label selector)
```

Lab Steps

01. Create a ClusterIP service

Creates a stable virtual IP that load-balances across all Pods with matching labels.

```
kubectl expose deploy/web --port=80 --target-port=80 --name=web-svc
kubectl get svc web-svc
```

Expected: ClusterIP assigned from 10.96.0.0/12 range.

Internals: Endpoints controller writes the Pod IPs into an Endpoints object with the same name as the Service.

02. Inspect Endpoints

Endpoints == the actual targets behind the virtual IP.

```
kubectl get endpoints web-svc
```

Expected: Comma-separated list of PodIP:80.

Internals: If this list is empty your selector is wrong or no Pod is ready.

03. Service discovery via DNS

DNS resolves the service name to its ClusterIP; HTTP returns the nginx welcome page.

```
kubectl run probe --rm -it --image=busybox -- sh -c 'nslookup web-svc.default.svc.cluster.local; wget -qO- we
```

Expected: Address: 10.96.x.x, then `<!DOCTYPE html>`.

Internals: CoreDNS holds a zone for cluster.local and answers from the in-memory Service/Endpoints cache.

04. Promote to NodePort

Allocates a port in 30000-32767 on every node forwarding to the Service.

```
kubectl patch svc web-svc -p '{"spec":{"type":"NodePort"}}'
kubectl get svc web-svc
```

Expected: PORT(S) shows `80:3xxxx/TCP`.

Internals: kube-proxy adds an extra iptables rule chain for the node port.

05. Break the selector on purpose

Empty Endpoints == 503 / Connection refused from clients. Classic outage.

```
kubectl patch svc web-svc -p '{"spec":{"selector":{"app":"wrong"}}}'
kubectl get endpoints web-svc
```

Expected: Endpoints list becomes <none>.

Internals: Endpoints controller can find no Pod matching the new selector.

06. Fix it

Restoring the right selector immediately repopulates Endpoints.

```
kubectl patch svc web-svc -p '{"spec":{"selector":{"app":"web"}}}'
```

Expected: Endpoints list filled again.

Internals: No Pod restart needed — only the Service object changed.

Common Errors

503 from a healthy Pod — Cause: Selector mismatch → empty Endpoints · Fix: kubectl get endpoints <svc>

DNS lookup times out — Cause: CoreDNS down or NetworkPolicy blocking egress to kube-dns · Fix: Check coredns pods and any default-deny NetworkPolicy

Troubleshooting

• Endpoints empty → selector or readiness

• Endpoints full but no connectivity → kube-proxy or CNI

• DNS slow → CoreDNS HPA + ndots:5

Enterprise Relevance

Service mesh, Ingress and LoadBalancer all sit on top of these primitives. Get them wrong and everything downstream is unreliable.

Validation

• You can reach a Pod by service DNS name

• You can detect a selector mismatch in <30s using `kubectl get endpoints`

Related Incidents & Guide Sections

Incidents: K8S-INC-3003

• Networking & Service types

• Service unreachable

Questions

Beginner

Q. What's the DNS name of a Service called `web-svc` in namespace `default`?

A. web-svc.default.svc.cluster.local

Intermediate

Q. What populates the Endpoints object?

A. The endpoints controller, based on the Service selector + Pod readiness.

Advanced

Q. Why use IPVS over iptables for kube-proxy?

A. O(1) rule lookup at large scale vs O(n) for iptables.

Scenario

Q. All clients report 503 but Pods are Running. First check?

A. `kubectl get endpoints <svc>` — if empty, look at selector/readiness.

Viva

Q. Difference between Service and Ingress?

A. Service = L4 in-cluster VIP. Ingress = L7 HTTP router, usually exposed via a single LoadBalancer.

Lab 06 — Flannel, CNI & Pod Networking

ID: K8S-LAB-06 · **Category:** Networking · **Level:** Engineer · **Duration:** 60 min

Focus: CNI, Flannel, PodCIDR, overlay, VXLAN

Objective

Understand what CNI actually does, inspect Flannel's pod, routes and config files, and recover from PodCIDR mismatch.

Concepts Covered

• The CNI spec (ADD/DEL/CHECK)

• VXLAN overlay basics

• Per-node /24 subnet from the cluster PodCIDR

Prerequisites

• A cluster from Lab 1 with Flannel

Architecture

Flannel gives every node a /24 from the PodCIDR, plumbs a cni0 bridge, and tunnels off-node traffic via VXLAN over UDP/8472.

```
Pod A (node1, 10.244.1.5)  <-- VXLAN -->  Pod B (node2, 10.244.2.7)
   |
 cni0 bridge
   |
 veth pair  --  Pod netns
```

Lab Steps

01. Inspect Flannel pods

One kube-flannel-ds Pod per node.

```
kubectl -n kube-flannel get pods -o wide
```

Expected: All Running 1/1.

Internals: Flannel runs as a DaemonSet so every node — including new ones added via kubeadm join — automatically gets a Flannel agent.

02. Inspect node subnet allocation

Shows which /24 this node owns.

```
cat /run/flannel/subnet.env
```

Expected: FLANNEL_SUBNET=10.244.0.1/24 (on the control plane).

Internals: Flannel allocates a subnet for the node via the node's spec.podCIDR field.

03. Inspect the CNI config drop

Without this file kubelet refuses to mark Pods Ready.

```
ls /etc/cni/net.d/
```

```
cat /etc/cni/net.d/10-flannel.conflist
```

Expected: JSON with plugins: [flannel, portmap].

Internals: kubelet picks the alphabetically first conflist as the active CNI.

04. Trace pod-to-pod traffic

Proves Pod-to-Pod connectivity across the CNI.

```
kubectl run a --image=busybox --command -- sleep 3600
kubectl run b --image=busybox --command -- sleep 3600
kubectl get pods -o wide
A_IP=$(kubectl get pod a -o jsonpath='{.status.podIP}')
kubectl exec b -- ping -c 3 $A_IP
```

Expected: 3 successful ICMP replies.

Internals: Same node → goes through cni0 bridge. Different nodes → goes through VXLAN tunnel.

05. Simulate a PodCIDR mismatch (don't run on production!)

Demonstrates the failure mode. Then revert.

```
kubectl -n kube-flannel edit configmap kube-flannel-cfg # change Network to 10.99.0.0/16
```

Expected: Flannel pods CrashLoopBackOff with `Network CIDR mismatch`.

Internals: Flannel's agent compares the ConfigMap to the node's podCIDR field. A mismatch is fatal.

Common Errors

Flannel CrashLoopBackOff — Cause: PodCIDR vs ConfigMap mismatch · Fix: Revert ConfigMap, or re-init the cluster with the matching `--pod-network-cidr`

CoreDNS stuck ContainerCreating — Cause: CNI config not written yet · Fix: Wait or `rollout restart` after CNI is healthy

Troubleshooting

• `ip route` on the node should list the per-node subnets via flannel.1

• `bridge link show` shows the cni0 bridge

Enterprise Relevance

Almost every cluster outage that 'feels like networking' is actually CNI. Knowing where /etc/cni/net.d/ and /run/flannel/ live saves hours.

Validation

• Pods can ping each other across nodes

• You can articulate what `subnet.env` is for

Related Incidents & Guide Sections

Incidents: K8S-INC-3003

• Networking & Service types

• CNI failure

Questions

Beginner

Q. What does CNI stand for?

A. Container Network Interface.

Intermediate

Q. Why does every node need its own /24?

A. So Pod IPs are unique and routable without coordination.

Advanced

Q. Why VXLAN over plain IP routing?

A. VXLAN works across L2 boundaries without needing the underlay to learn Pod CIDRs.

Scenario

Q. New node joins, pods on it can't reach pods elsewhere. First check?

A. Flannel pod on the new node, then VXLAN UDP/8472 on the underlay firewall.

Viva

Q. What does the kubelet check before marking a Pod Ready w.r.t. CNI?

A. That CNI ADD succeeded and an IP was returned for the sandbox.

Lab 07 — Resources, Requests, Limits & OOMKilled

ID: K8S-LAB-07 · **Category:** Core · **Level:** Engineer · **Duration:** 45 min

Focus: requests, limits, cgroups, OOMKilled, node allocatable

Objective

Configure resource requests/limits, observe scheduler decisions, and reproduce/recover from OOMKilled.

Concepts Covered

• Difference between requests and limits

• How requests influence scheduling

• How limits become cgroup constraints

Prerequisites

• A cluster from Lab 1

Architecture

Requests are the contract with the scheduler. Limits are the contract with the kernel.

```
requests  ->  scheduler reservation (fit decision)
limits    ->  cgroup hard cap (CPU throttle / OOMKill)
```

Lab Steps

01. Deploy a pod with explicit resources

Asks for 64Mi but tries to use 250Mi → guaranteed OOMKill.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata: {name: capped}
spec:
  containers:
  - name: app
    image: polinux/stress
    resources:
      requests: {memory: "64Mi", cpu: "100m"}
      limits:   {memory: "128Mi", cpu: "200m"}
    command: ["stress","--vm","1","--vm-bytes","250M","--vm-hang","1"]
EOF
```

Expected: STATUS goes Running → OOMKilled within ~10s.

Internals: Kernel OOM killer fires when the memory cgroup is over limit; kubelet reports OOMKilled in the Pod status.

02. Inspect the killed container

Shows Reason=OOMKilled and the exit code (137).

```
kubectl describe pod capped | grep -A2 'Last State'
```

Expected: Reason: OOMKilled, Exit Code: 137.

Internals: Exit code 137 = 128 + SIGKILL(9).

03. Right-size and re-deploy

Pods are mostly immutable; recreate with a higher limit.

```
kubectl patch pod capped --type='json' -p='[{"op":"replace","path":"/spec/containers/0/resources/limits/memor
kubectl delete pod capped && \
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: Pod
metadata: {name: capped}
spec:
  containers:
  - name: app
    image: polinux/stress
    resources:
      requests: {memory: "256Mi", cpu: "100m"}
      limits:   {memory: "512Mi", cpu: "200m"}
    command: ["stress","--vm","1","--vm-bytes","250M","--vm-hang","1"]
EOF
```

Expected: Pod stays Running.

Internals: Pod resources are immutable today; in-place resize is alpha behind a feature gate.

04. Inspect node allocatable

Shows what the scheduler thinks is available after system-reserved and kube-reserved.

```
kubectl describe node | grep -A5 Allocatable
```

Expected: CPU/memory values lower than total — that's expected.

Internals: Allocatable = capacity - kubeReserved - systemReserved - evictionThresholds.

Common Errors

OOMKilled in a loop — Cause: Limit too low for the actual working set · Fix: Profile, then raise limit (or fix the leak)

Pod won't schedule — Cause: requests > node allocatable · Fix: Lower requests, or add a node

Troubleshooting

• `kubectl top pod` to see usage if metrics-server is installed

• `dmesg | grep -i oom` on the node confirms kernel OOM

Enterprise Relevance

Mis-set limits cause the majority of mystery production restarts. Every workload spec needs explicit requests and limits.

Validation

• You can reproduce OOMKilled and recover from it

• You can read `Allocatable` and predict whether a Pod will fit

Related Incidents & Guide Sections

Incidents: K8S-INC-3001

• Pod lifecycle & probes

• OOMKilled

Questions

Beginner

Q. What's exit code 137?

A. 128 + SIGKILL(9) — the kernel OOM killer.

Intermediate

Q. Why doesn't setting only limits help the scheduler?

A. The scheduler only looks at requests.

Advanced

Q. What's the difference between system-reserved and kube-reserved?

A. system-reserved is for the OS (sshd etc.); kube-reserved is for kubelet/runtime/cni.

Scenario

Q. A new microservice OOMs every few hours. Approach?

A. Add usage metrics → compute p95 working set → set limit ~1.5x p95, then iterate.

Viva

Q. What QoS class does a Pod get if requests==limits?

A. Guaranteed.

Lab 08 — Persistent Storage — PV, PVC & StorageClass

ID: K8S-LAB-08 · **Category:** Core · **Level:** Engineer · **Duration:** 45 min

Focus: PersistentVolume, PersistentVolumeClaim, StorageClass, dynamic provisioning

Objective

Provision storage dynamically with a StorageClass, attach it to a Pod, and prove the data survives Pod deletion.

Concepts Covered

• The PV/PVC binding loop

• Reclaim policies (Retain vs Delete)

• Access modes (RWO/ROX/RWX)

Prerequisites

• A cluster with a default StorageClass (kind/minikube ship one; bare kubeadm needs local-path-provisioner)

Architecture

PVCs are the consumer-facing API. PVs are the supply. A StorageClass automates the supply by calling a CSI driver.

Lab Steps

01. Install local-path-provisioner (if no default class)

Gives the cluster a default StorageClass backed by local disk.

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/master/deploy/local-path-st
kubectl patch storageclass local-path -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-
```

Expected: storageclass.storage.k8s.io/local-path created.

Internals: The provisioner watches PVCs and creates a hostPath PV on demand.

02. Create a PVC and a Pod that uses it

PVC requests 1Gi. The provisioner creates a matching PV. Pod mounts it at /data.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: data}
spec:
  accessModes: [ReadWriteOnce]
  resources: {requests: {storage: 1Gi}}
```

```
---
apiVersion: v1
kind: Pod
metadata: {name: writer}
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","echo hello > /data/file; sleep 3600"]
    volumeMounts: [{mountPath: /data, name: vol}]
  volumes:
  - name: vol
    persistentVolumeClaim: {claimName: data}
EOF
```

Expected: PVC STATUS=Bound, Pod Running.

Internals: PV/PVC binding is atomic; once bound a PVC can only mount that PV.

03. Prove persistence

Deleting the Pod doesn't delete the PVC or its data.

```
kubectl exec writer -- cat /data/file
kubectl delete pod writer
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: Pod
metadata: {name: reader}
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","cat /data/file && sleep 3600"]
    volumeMounts: [{mountPath: /data, name: vol}]
  volumes:
  - name: vol
    persistentVolumeClaim: {claimName: data}
EOF
kubectl logs reader
```

Expected: `hello` printed by the second Pod.

Internals: Reclaim policy on dynamically-provisioned PVs defaults to Delete — but only triggers when the PVC itself is deleted.

Common Errors

PVC stuck Pending — Cause: No StorageClass / no PV matches · Fix: kubectl describe pvc to see the binder error

Pod can't mount volume — Cause: RWO PV already bound elsewhere, or node mismatch · Fix: Check PV claimRef and topology constraints

Troubleshooting

• `kubectl get pv,pvc -A`

• Check the provisioner pod's logs

Enterprise Relevance

Stateful workloads live or die by correct PVC sizing and reclaim policy. Wrong reclaim = data loss.

Validation

• Data outlives the Pod that wrote it

• You can name the three access modes

Related Incidents & Guide Sections

Incidents: K8S-INC-3004

• Storage / PV / PVC / CSI

• PVC Pending

Questions

Beginner

Q. What does a PVC do?

A. Requests storage matching a size + access mode.

Intermediate

Q. When does Reclaim=Delete actually delete?

A. When the PVC is deleted; deleting the Pod alone does not trigger it.

Advanced

Q. Why is RWX rare on block storage?

A. Block devices generally don't support multi-writer; you need a network filesystem.

Scenario

Q. PVC stuck Pending forever. What's the most likely cause?

A. No StorageClass or no provisioner — kubectl describe pvc shows the binder error.

Viva

Q. Why prefer StorageClass over manually creating PVs?

A. Dynamic provisioning — scales with demand, no manual ops.

Lab 09 — RBAC, Secrets, ServiceAccounts & NetworkPolicies

ID: K8S-LAB-09 · **Category:** Enterprise · **Level:** Enterprise · **Duration:** 60 min

Focus: RBAC, Roles, RoleBindings, Secrets, ServiceAccounts, NetworkPolicy

Objective

Lock down a namespace with least-privilege RBAC, mount a Secret safely, and apply a default-deny NetworkPolicy.

Concepts Covered

• Role vs ClusterRole

• ServiceAccount tokens

• Why Secrets aren't really 'secret' by default

• Default-deny network posture

Prerequisites

• A cluster from Lab 1

Architecture

RBAC is just three objects: Role describes *what*, RoleBinding describes *who*, ServiceAccount provides the identity.

```
Subject (User/SA/Group)  --(RoleBinding)-->  Role  --(rules)-->  Resources
```

Lab Steps

01. Create a namespace and a junior SRE ServiceAccount

Identity for our least-privilege actor.

```
kubectl create namespace ops
kubectl -n ops create serviceaccount sre-junior
```

Expected: namespace/ops and serviceaccount/sre-junior created.

Internals: Every namespace also gets a default ServiceAccount automatically.

02. Define a read-only Role

Lets sre-junior read pods/logs/events in `ops` only.

```
cat <<EOF | kubectl -n ops apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: read-only}
rules:
- apiGroups: [""]
  resources: ["pods","pods/log","events"]
  verbs: ["get","list","watch"]
EOF
kubectl -n ops create rolebinding sre-junior-read --role=read-only --serviceaccount=ops:sre-junior
```

Expected: role and rolebinding created.

Internals: RBAC is additive and namespaced unless a ClusterRole/ClusterRoleBinding is used.

03. Test as the ServiceAccount

auth can-i is the fastest way to verify RBAC.

```
kubectl -n ops auth can-i list pods --as=system:serviceaccount:ops:sre-junior
kubectl -n ops auth can-i delete pods --as=system:serviceaccount:ops:sre-junior
```

Expected: yes / no.

Internals: API server simulates the RBAC check with the requested identity.

04. Create and mount a Secret

Secret is injected via env. Logs show the value — proving the mount, but also why Secrets need RBAC + encryption-at-rest.

```
kubectl -n ops create secret generic db --from-literal=password=s3cret
kubectl -n ops run app --image=busybox --restart=Never -- sh -c 'echo $DB_PASS; sleep 3600' --overrides='{
  "spec": {
    "containers": [{
      "name":"app","image":"busybox",
      "command":["sh","-c","echo $DB_PASS; sleep 3600"],
      "env":[{"name":"DB_PASS","valueFrom":{"secretKeyRef":{"name":"db","key":"password"}}}]
    }]
  }
}'
kubectl -n ops logs app
```

Expected: `s3cret`

Internals: Secrets are base64-encoded, not encrypted, by default. Enable EncryptionConfiguration on the API server for real protection.

05. Apply default-deny ingress and re-allow only what's needed

All inbound traffic to Pods in `ops` is now blocked unless explicitly allowed.

```
cat <<EOF | kubectl -n ops apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: default-deny-ingress}
spec:
  podSelector: {}
  policyTypes: [Ingress]
EOF
```

Expected: networkpolicy.networking.k8s.io/default-deny-ingress created.

Internals: Requires a CNI that enforces NetworkPolicy (Calico/Cilium). Flannel needs the kube-router or canal variant.

Common Errors

NetworkPolicy has no effect — Cause: CNI doesn't enforce them (e.g. vanilla Flannel) · Fix: Use Calico/Cilium or add kube-router

Forbidden errors even after RoleBinding — Cause: Wrong namespace or wrong subject kind · Fix: auth can-i --as=... pinpoints it

Troubleshooting

• `kubectl auth can-i --list -n ops --as=...` enumerates effective rights

• `kubectl get rolebinding,clusterrolebinding -A | grep <sa>`

Enterprise Relevance

RBAC + NetworkPolicy + encrypted Secrets is the minimum bar for a compliant cluster (SOC2, ISO27001, PCI).

Validation

• sre-junior can read but not write

• Default-deny NetworkPolicy is active in `ops`

Related Incidents & Guide Sections

Incidents: K8S-INC-3005

• RBAC & PodSecurityAdmission

Questions

Beginner

Q. What is a ServiceAccount for?

A. An identity for in-cluster workloads to authenticate to the API.

Intermediate

Q. Difference between Role and ClusterRole?

A. Role is namespaced; ClusterRole is cluster-wide or used for non-namespaced resources.

Advanced

Q. Why do default Secrets not satisfy compliance?

A. They're only base64 in etcd unless EncryptionConfiguration is enabled.

Scenario

Q. A pod is exfiltrating data. How do you block egress immediately?

A. NetworkPolicy with Egress type and an empty allow list, scoped to the pod's labels.

Viva

Q. Walk through `kubectl auth can-i`.

A. Sends a SubjectAccessReview to the API server with the supplied user; returns the RBAC decision.

Lab 10 — Monitoring & Observability — Prometheus, Grafana, Loki

ID: K8S-LAB-10 · **Category:** Enterprise · **Level:** Enterprise · **Duration:** 75 min

Focus: Prometheus, Grafana, Loki, metrics-server, alerting

Objective

Install the kube-prometheus-stack, visualize cluster metrics in Grafana, ship logs to Loki, and write a basic PromQL alert.

Concepts Covered

• Pull-based metrics vs push

• Service discovery via ServiceMonitor

• Log labels vs full-text indexing

Prerequisites

• A cluster from Lab 1

• Helm 3 installed

Architecture

Prometheus scrapes; Loki indexes labels not log lines (cheaper); Grafana stitches both into a single dashboard.

```
Pods --(/metrics)--> Prometheus  ---->  Grafana
            (kubelet/cadvisor)
Pods --(stdout)--->  Promtail  ---->  Loki  ---->  Grafana
```

Lab Steps

01. Install kube-prometheus-stack

One chart ships Prometheus, Grafana, Alertmanager, node-exporter and a pile of dashboards.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
```

Expected: All pods Running 1/1 in `monitoring` within ~2 min.

Internals: ServiceMonitor CRDs are how Prometheus discovers what to scrape inside the cluster.

02. Open Grafana

Default admin password is `prom-operator` for the chart.

```
kubectl -n monitoring port-forward svc/monitoring-grafana 3000:80
# in browser: http://localhost:3000 (admin / prom-operator)
```

Expected: Grafana UI loads with prebuilt 'Kubernetes / Compute Resources' dashboards.

Internals: Port-forward tunnels via the API server; no LoadBalancer needed for the lab.

03. Install Loki and Promtail

Promtail (DaemonSet) ships /var/log/containers/* to Loki; Grafana already discovers Loki via the same release.

```
helm install loki prometheus-community/loki-stack -n monitoring
```

Expected: loki-* pods Running.

Internals: Promtail tails container log files and adds Pod labels as Loki labels.

04. Write a PromQL alert

Fires when any pod restarts more than 6×/hour for 5 minutes.

```
cat <<EOF | kubectl -n monitoring apply -f -
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata: {name: pod-restarts, labels: {release: monitoring}}
spec:
  groups:
  - name: pods
    rules:
    - alert: PodRestartingTooMuch
      expr: rate(kube_pod_container_status_restarts_total[5m]) > 0.1
      for: 5m
      labels: {severity: warning}
      annotations: {summary: "Pod {{ \$labels.pod }} restarting"}
EOF
```

Expected: prometheusrule.monitoring.coreos.com/pod-restarts created.

Internals: The Prometheus Operator reloads Prometheus when PrometheusRule CRs change.

Common Errors

No data in Grafana — Cause: ServiceMonitors filtered by `release` label mismatch · Fix: Add `labels: {release: monitoring}` to your ServiceMonitor

Loki disk fills fast — Cause: Default retention too long for log volume · Fix: Tune `limits_config.retention_period`

Troubleshooting

• `kubectl -n monitoring get servicemonitor`

• Prometheus → Status → Targets to see who's being scraped

Enterprise Relevance

You cannot run Kubernetes blind. The stack above is the lingua franca; nearly every SRE job description assumes it.

Validation

• You can see CPU/memory per Pod in Grafana

• You can query logs by Pod label in Loki

• You wrote one alert

Related Incidents & Guide Sections

Incidents: K8S-INC-3001, K8S-INC-3004

• Observability stack

Questions

Beginner

Q. Why pull instead of push for metrics?

A. Easier service discovery and built-in liveness signal.

Intermediate

Q. What's a ServiceMonitor?

A. A CRD the Prometheus Operator turns into a scrape config.

Advanced

Q. Why does Loki index labels not log content?

A. Drastically lower storage and cost; you grep within the time-bounded label stream.

Scenario

Q. Prometheus disk fills every weekend. What do you tune?

A. Retention, scrape interval, drop noisy series with relabel rules.

Viva

Q. Difference between recording and alerting rules?

A. Recording precomputes a series; alerting fires when an expr stays true for `for`.

Lab 11 — Hybrid Cluster — Joining a Windows Worker Node

ID: K8S-LAB-11 · **Category:** Windows · **Level:** Enterprise · **Duration:** 90 min

Focus: Windows Kubernetes, kubelet.exe, HNS, containerd-windows, hybrid scheduling

Objective

Add a Windows Server 2022 node to a Linux control plane, schedule an IIS Pod on it, and debug HNS networking.

Concepts Covered

• Why Windows nodes need their own CNI (Calico-Windows or Flannel host-gw)

• OS-based nodeSelectors and tolerations

• HNS endpoints vs Linux veth pairs

Prerequisites

• Lab 1 cluster

• Windows Server 2022 VM with 4 vCPU / 8 GB RAM, joined to the same L2/L3 as the Linux node

Architecture

Same control plane, two node OS families. The scheduler uses kubernetes.io/os labels and nodeSelectors to keep Linux Pods on Linux nodes and Windows Pods on Windows nodes.

Lab Steps

01. Prepare the Windows node

Containers + Hyper-V are required for containerd-windows and HNS.

```
# On the Windows node (PowerShell as admin):
Install-WindowsFeature -Name Containers
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All -NoRestart
Restart-Computer
```

Expected: Server reboots, features enabled.

Internals: HNS (Host Network Service) is the Windows equivalent of cni0 + iptables. It's a Hyper-V virtual switch.

02. Install containerd, kubelet.exe, kubectl.exe

PrepareNode.ps1 installs the kubelet service, kubectl, and Calico/Flannel binaries.

```
# PowerShell:
```

```
curl.exe -LO https://github.com/containerd/containerd/releases/download/v1.7.13/containerd-1.7.13-windows-amd
tar -xvf containerd-1.7.13-windows-amd64.tar.gz -C $env:ProgramFiles\containerd
& $env:ProgramFiles\containerd\bin\containerd.exe --register-service
Start-Service containerd
# install kubelet & kubectl via SIG-Windows scripts:
curl.exe -LO https://raw.githubusercontent.com/kubernetes-sigs/sig-windows-tools/master/PrepareNode.ps1
.\PrepareNode.ps1 -KubernetesVersion v1.30.0
```

Expected: Service `kubelet` registered and Running.

Internals: kubelet.exe behaves like Linux kubelet but talks to containerd via the Windows named-pipe CRI socket.

03. Join the node

Same join command as Linux, just kubeadm.exe.

```
# Re-run `kubeadm token create --print-join-command` on the LINUX control plane,
# then on Windows in an elevated shell:
kubeadm.exe join <CP_IP>:6443 --token <t> --discovery-token-ca-cert-hash sha256:<h>
```

Expected: `This node has joined the cluster`

Internals: TLS bootstrap is identical; only the kubelet binary differs.

04. Verify and schedule an IIS pod

nodeSelector pins the Pod to Windows. The IIS image is ~5 GB so the first pull is slow.

```
kubectl get nodes -o wide
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: iis}
spec:
  replicas: 1
  selector: {matchLabels: {app: iis}}
  template:
    metadata: {labels: {app: iis}}
    spec:
      nodeSelector: {kubernetes.io/os: windows}
      tolerations:
      - {key: "os", operator: "Equal", value: "windows", effect: "NoSchedule"}
      containers:
      - name: iis
        image: mcr.microsoft.com/windows/servercore/iis:windowsservercore-ltsc2022
        ports: [{containerPort: 80}]
EOF
kubectl get pods -o wide -l app=iis
```

Expected: Pod Running on the Windows node within ~10 min on first pull.

Internals: Windows containers are NOT lightweight — image layers are huge because each carries a chunk of the Windows base image.

Common Errors

Windows node NotReady — Cause: CNI (Calico/Flannel-Windows) not installed · Fix: Run sig-windows install-cni.ps1

Pod stuck ContainerCreating on Windows — Cause: HNS endpoint creation failed · Fix: Restart-Service hns; check `Get-HnsNetwork`

ImagePullBackOff for `mcr.microsoft.com/windows/...` — Cause: Pulled to a Linux node by accident · Fix: Add nodeSelector kubernetes.io/os=windows

Troubleshooting

• `Get-HnsNetwork`, `Get-HnsEndpoint`

• Windows event log: kubelet writes to Application

• `crictl --runtime-endpoint npipe:////./pipe/containerd-containerd ps`

Enterprise Relevance

Mixed Linux/Windows clusters are common in enterprises with .NET Framework legacy. Modernization rarely deletes Windows workloads — it just orchestrates them.

Validation

• A Windows Pod runs alongside Linux Pods in the same cluster

• You can reach the IIS Pod's PodIP from a Linux Pod

Related Incidents & Guide Sections

Incidents: K8S-INC-3002

• Linux ↔ Windows hybrid

• Join Windows node

Questions

Beginner

Q. What label distinguishes a Windows node?

A. kubernetes.io/os=windows

Intermediate

Q. Why is the IIS image multi-GB?

A. It bundles the Windows Server Core base image — there's no shared host kernel.

Advanced

Q. What does HNS do?

A. Provides the virtual switch and endpoints that map to Pod network namespaces on Windows.

Scenario

Q. All Windows pods go NotReady at once. First check?

A. Get-Service hns on each Windows node, then the Windows CNI pod.

Viva

Q. Can a Pod contain both a Linux and a Windows container?

A. No — a Pod runs on a single node, which has a single OS.

Lab 12 — Enterprise Incident Investigation Capstone

ID: K8S-LAB-12 · **Category:** Investigation · **Level:** Enterprise · **Duration:** 120 min

Focus: P1 incident response, root cause, recovery, postmortem, blast radius

Objective

Run an end-to-end P1 investigation: alert → triage → root cause → recovery → validation → postmortem note.

Concepts Covered

• Investigation playbooks

• Blast radius reasoning

• Recovery vs root cause (fix forward vs rollback)

• Communicating during a live incident

Prerequisites

• All previous labs

Architecture

Real incidents are not a single command. They're a loop you exit only when metrics, logs and customer reports all agree the issue is gone.

```
ALERT  ->  Triage  ->  Hypothesis  ->  Test  ->  Recover  ->  Validate  ->  Postmortem
           (5-min)  (15-min)         (loop)       (act)      (prove)      (next day)
```

Lab Steps

01. Pick an incident from the ops console

Choose any K8S-INC-30xx. We'll walk through K8S-INC-3001 (CrashLoopBackOff cascade).

```
# Open /ops/k8s-incidents in the ShadowXLab app.
```

Expected: Incident detail page open.

Internals: Incidents in the lab are deterministic — symptoms map 1:1 to a known root cause.

02. Triage (5 minutes)

Three commands that give you the cluster-wide pulse.

```
kubectl get pods -A --field-selector=status.phase!=Running | head
kubectl top pods -A | sort -k4 -h | tail
kubectl get events -A --sort-by=.lastTimestamp | tail -20
```

Expected: List of unhealthy pods + top consumers + recent events.

Internals: These are the closest equivalents to `dmesg`, `top` and `journalctl -f` for a cluster.

03. Form a hypothesis

A written hypothesis stops you flailing.

```
# Document in the incident ticket: 'Suspect probe regression after image bump'.
```

Expected: Hypothesis in the ticket.

Internals: Postmortems judge investigations on the *quality* of the hypothesis tree, not the time-to-fix.

04. Test and recover

Rollback is the cheapest test: if the previous revision works, the new one introduced the regression.

```
kubectl rollout history deploy/<svc>
kubectl rollout undo deploy/<svc>
kubectl rollout status deploy/<svc>
```

Expected: Service back to healthy.

Internals: Always recover first, root-cause second. Customers don't care about the why.

05. Validate

Validation = 'metrics agree AND no new errors AND a synthetic transaction succeeds'.

```
# Re-run the triage commands; confirm SLI dashboards back to green.
kubectl get pods -A --field-selector=status.phase!=Running | wc -l
```

Expected: Unhealthy pod count returns to baseline; Grafana SLI ≥ target.

Internals: Without explicit validation, you'll be paged again in 10 minutes.

06. Write the postmortem

Action items must be specific, owned and dated. Otherwise the incident repeats.

```
# Use the template: Timeline | Impact | Root cause | What went well | What didn't | Action items.
```

Expected: Document attached to the incident ticket.

Internals: Blameless postmortems are mandatory in mature SRE orgs; they convert outage cost into learning value.

Common Errors

Tunnel vision — Cause: Chasing the first hypothesis past the point of evidence · Fix: Time-box hypotheses to 15 minutes; if no progress, regroup

Root-cause before recovery — Cause: Engineer's instinct · Fix: Stop the bleeding first

Troubleshooting

• Always have a written hypothesis

• Always have an explicit recovery action

• Always validate before declaring resolution

Enterprise Relevance

This loop is the difference between a senior SRE and a button-pusher. It scales from a 1-pod misconfig to a multi-region outage.

Validation

• Service restored to baseline SLI

• Postmortem published within 24h

• At least one preventive action item filed

Related Incidents & Guide Sections

Incidents: K8S-INC-3001, K8S-INC-3002, K8S-INC-3003, K8S-INC-3004, K8S-INC-3005

• Troubleshooting methodology

• Enterprise patterns

Questions

Beginner

Q. What's the first goal of an incident response?

A. Stop customer impact (recover), not find root cause.

Intermediate

Q. Why time-box hypotheses?

A. Avoid tunnel vision; force re-evaluation against fresh evidence.

Advanced

Q. Why are blameless postmortems more valuable than blame ones?

A. They surface systemic issues; blame just hides them.

Scenario

Q. 30% of pods OOMKill after a deploy. You can't reach the dev who pushed it. Action?

A. Rollback the deploy; root-cause and contact dev after restoration.

Viva

Q. Walk me through your last (or hypothetical) P1.

A. Use the loop: triage → hypothesis → test → recover → validate → postmortem.